The Governed Data Product for AI

How eight raw banking files became data an AI assistant can query correctly — and only for the clients each user may see

Berka (PKDD'99) take-home · technical walkthrough · 9 November 2026

The brief, and the one idea behind my answer

Task 1 · Data quality

Rules for completeness, uniqueness, referential integrity, validity, consistency — each with a severity and a reason.

Task 2 · AI-ready product

A model, a semantic layer, and an LLM answering ten questions — generated SQL and result per question.

Task 3 · Entitlements

Each user sees only their own clients. Where the filter sits, why the LLM cannot bypass it, onboarding without code.

Treat the AI assistant as a dumb and dangerous client

Dumb — it guesses at anything the data does not spell out. Every layer should remove a reason to guess.

Dangerous — whatever it writes runs against customer data. Every layer should remove a way to do harm.

Six schemas in one PostgreSQL database

raw

BRONZE

8 files verbatim as text, SHA-256 per file, line numbers

stg

SILVER

Typed, decoded, sign resolved — nothing filtered, originals kept

dq

CONTROL

124 rules, quarantine, flags, reconciliation, run log

core

GOLD

Star schema = stg − quarantine, verified each build

ai

SURFACE

11 business-named views, all security_invoker

sec

ACCESS

Principals, grants, TOTP — read by 7 RLS policies

Contract	Checked by	Delivered result
raw = stg on rows and money	DQ step (dq.reconciliation)	11 of 11 balanced
core = stg − quarantine on rows and money	build_model.reconcile_core	9 of 9 balanced, every delta 0
semantic layer = live schema	make semantic	0 problems (11 entities, 105 columns, 19 joins)

How I worked: rules that explain most decisions

Executed, not asserted

Every number comes from a logged command; a step that did not run says NOT EXECUTED

Land verbatim, never repair

Originals survive in _raw columns with a note (D013)

Grade, do not gate

Staging filters nothing; rules decide afterwards (D020)

A severity has a consequence

block quarantines · warn flags travel to the answer · info logs

One definition, one place

Owner, problematic, incoming, month-end decided once (D006, D017)

Distrust your own build

Model verified every build; 20 seeded defects; security tested by logging in

1 Ingest: land everything exactly as it arrived

8

original .asc files from a verified public mirror (D003)

1,079,680

rows landed as text, file + line on every row (C01, C02)

SHA-256

per file in raw.load_audit — proof of which bytes

8 / 8

row counts match the published reference

Why everything lands as text

A cast at load time destroys the evidence of the defect it trips over: a bad date aborts the load or vanishes. Bronze is the one layer that cleans nothing, so when a number is disputed I can show what actually arrived.

Why defects are injected elsewhere (D004)

The public copy is clean enough that most rules never fire — and a rule that never fired proves nothing. A seeded, corrupted copy in its own schema proves the rules while the ten answers stay on untouched data.

2 Profile, then clean without overwriting

21 cleaning steps — exactly one removes a row

Step	What it does	Measured
C03	Seven spellings of 'missing' ('', ?, NA, N/A, NULL, -, unknown) become NULL; original kept	1,380 rows
C05	YYMMDD text becomes a date; two-digit years read as 19xx; a failed parse is NULL, not an error	1,062,394 dates
C07–C08	Birth date and gender decoded from birth_number (women: month + 50) — labelled derived	5,369 clients
C09	Czech codes decoded to English labels from one map; the code stays beside the label	2,466,175 values
C10	Sign resolved once: PRIJEM credit; VYDAJ and VYBER debit (VYBER is not in the data description)	1,056,320 postings
C14	The one row removed: a duplicate posting, quarantined with its reason	1 row

Profiling ran first: every column measured and 17 candidate defects written down with counts before any rule existed.

3 Grade every defect — and give the grade a consequence

BLOCK

Quarantined, excluded from core and the ai views, reversible

1 rule · 1 row

WARN

Kept; the flag travels onto the row and into the semantic layer as a caveat

6 rules · 440 flags

INFO

Standardised or logged; no answer changes

12 rules

124 rules (89 hand written + 35 completeness sweep) · 19 fire on the public copy · 14 defect types

Same check, opposite decision

14 postings have amount zero. 'Amount must be positive' sounds right — but they are real interest runs that accrued nothing; blocking them would break 14 balance chains to fix nothing, so they are info. A negative amount would be block: the sign was applied twice and money moves the wrong way.

4 Prove the rules fire, and prove nothing was lost

20 of 20 seeded defects caught

A deterministic injector corrupts a copy of staging in its own schema, one defect per rule family; a test asserts each is caught by the rule meant to catch it.

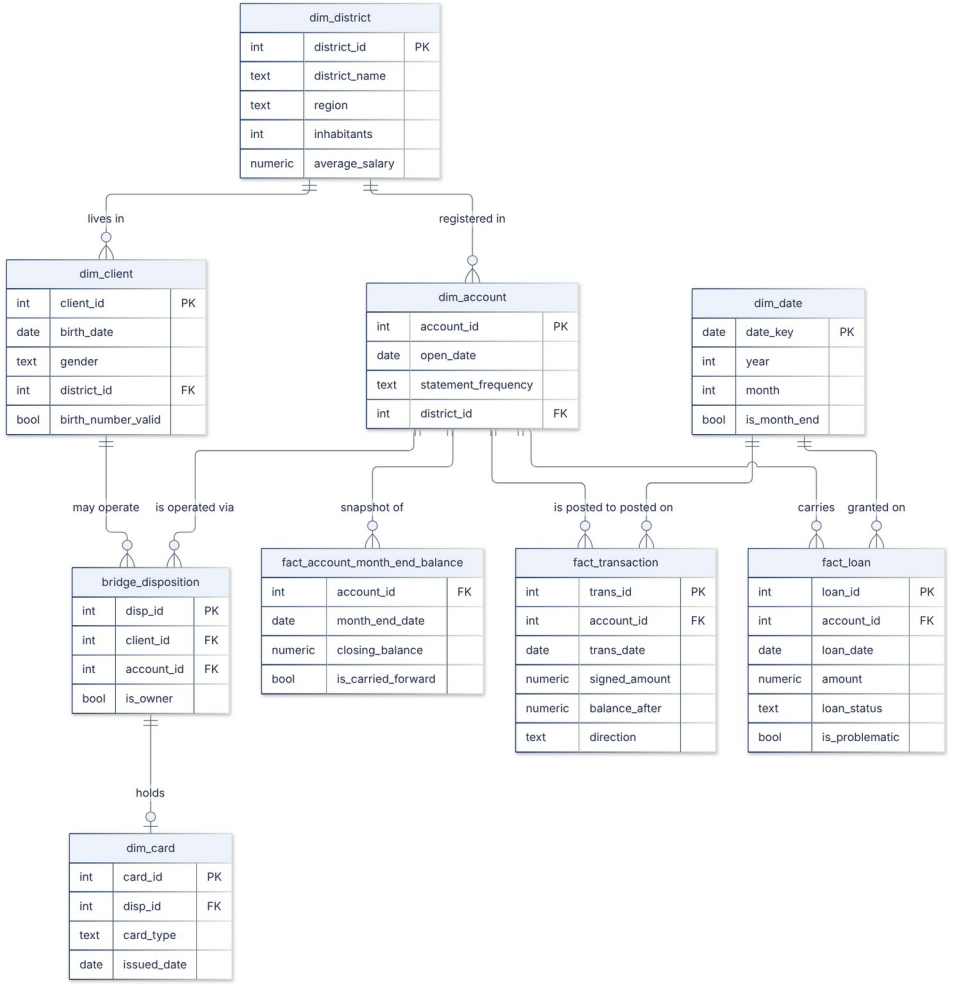
Reconciled after every build

raw = stg on 8 row counts and 3 money sums. core = stg − quarantine on 7 row counts and 2 money sums (transactions 6,257,793,560.30; loans 103,261,740.00).

The bug that proved the point

The quarantine view was not scoped by schema, so running the rule tests made the injected defects exclude real rows: 76 districts instead of 77, 11,024 transactions orphaned. The model's grain and foreign-key checks failed before any deliverable was written (D008). A green build is not a correct build.

5 A star schema with one definition per term



Term	Decided once in	Definition
Account owner	core.v_account_owner	The OWNER disposition — exactly one per account (block rule)
Problematic loan	fact_loan.is_problematic	Status B or D: 76 of 682 = 11.14% (A4)
Incoming money	fact_transaction.direction	Credits (PRIJEM), unsigned amount (A7)
Month-end balance	fact_account_month_end_balance	Last posting on or before month end; carried forward if none (D019)

Why a star with a bridge

Four grains (client, account, loan, posting); a wide table turns every count into count(distinct), which generated SQL gets wrong quietly. One source, no history: natural keys, no Data Vault (D007).

6 A semantic layer the build can refuse

What the model is given — and nothing else

- 11 entities with grain, key and every column described
- 12 certified metrics with their time column
- 19 joins with measured cardinality
- A glossary of what each term is and is not
- 7 data quality caveats with row counts
- 12 query rules (ai only, one SELECT, half-open dates, never aggregate after a fan-out, say what is missing instead of guessing)

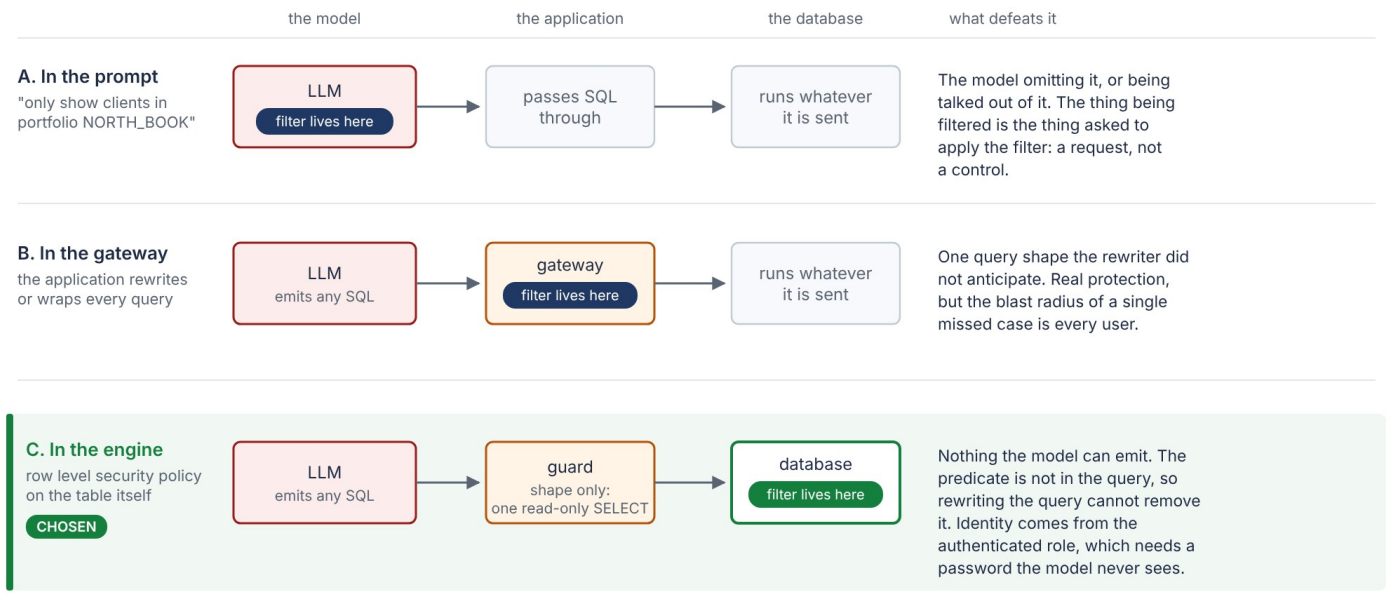
The build fails if ...

- a described column does not exist, or a column is undescribed
- a grain is not one row per key
- a join's cardinality differs from the declared one
- a metric names a column that does not exist

Why a file, not prompt text

A layer that drifted from the schema gives the model confident wrong information it cannot check.

7 Where the filter sits — and why the model cannot remove it



Logged in as	Clients	Loans
cro_global (portfolio of every client)	5,369	682
rm_north	1,481	179
rm_south (book + 1 delegated)	1,387	195
rm_unassigned (no portfolio)	0	0

Where

7 RLS policies on the client-bearing core tables; every view between is security_invoker

Why no bypass

The predicate is not in the model's text; identity is session_user; no consumer has BYPASSRLS

Onboarding

Insert principal + portfolio, create the role, enrol TOTP. Offboarding: one UPDATE

8 Access is data: four sources, two factors

Ownership	*derived*	view + operate on the client and their own accounts
Disposition	*derived*	that one account only — never the owner (D024)
Portfolio	*derived*	view only on the book (accountability ≠ payment authority)
Delegation	*stored*	a human decision with grantor, reason and expiry

Two factors, enforced by the database

Password, then a TOTP code verified inside PostgreSQL by a function that takes only the code and uses session_user. Single use; five committed failures lock for 15 minutes. Password alone → 0 rows (D026).

Limit found at delivery

A direct client can roll back failed guesses and is never locked out. Recorded as a strict expected failure; RLS unaffected.

Grants are derived live (D027): a materialised version once let a new portfolio do nothing until the next rebuild.

Defence in depth around a dumb and dangerous client

Layer	Enforces	Does not
Isolation	No tools, no files, no database; one turn, one JSON object	make the model right
Semantic layer	Meaning: certified metrics, 'is not' glossary, caveats, query rules	check the model followed it
SQL guard (sqlglot)	One read-only SELECT on ai; no out-of-schema functions; LIMIT ≤ 10,000	**resolve columns or judge meaning**
Execution envelope	Read-only transaction, 60 s timeout, at most cap + 1 rows, the user's own 2FA session	—
Row level security	Which rows exist for this user, whatever the SQL says	—
PostgreSQL	An unknown name is an error, recorded — never answered	catch a valid but wrong query
Evaluation	Value-based comparison with independent reference SQL	protect a live user

The guard proves a query is safe to run. It does not prove it answers the question.

Safe is not the same as correct

“Find male or female loans in Q1 1995.”

```
SELECT COUNT(*) AS total_loans,
  SUM(CASE WHEN T2.gender = 'male' THEN 1 ELSE 0 END) …
FROM ai.loan T1
JOIN ai.account_owner T2 ON T1.account_id = T2.account_id
WHERE T1.loan_date >= '1995-01-01'
  AND T1.loan_date <  '1995-04-01'
```

GUARD: ALLOWED

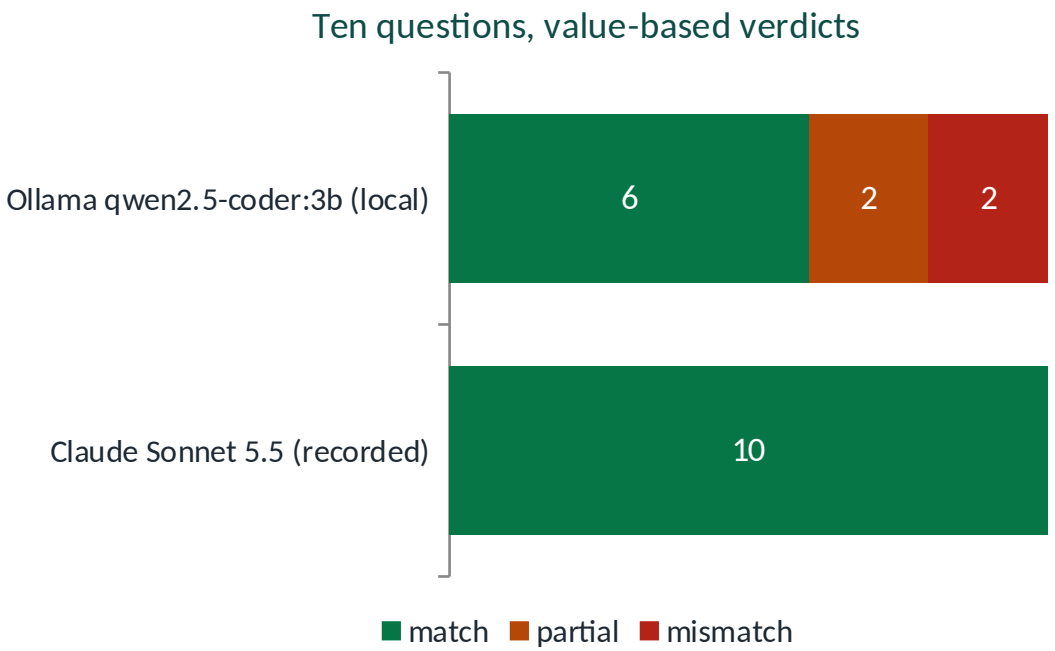
DATABASE: column t2.gender does not exist

Gender lives on ai.client; ai.loan and ai.account_owner expose owner_gender. The metric text had said 'any entity with a gender column'. It failed safely — the dangerous case is valid SQL that is wrong.

A small local model, same layer

- Q10: counted problematic loans instead of a share → 'monthly' (right: after transaction, 18.75%)
- Q6: joined to every day of the month → 279 new accounts in January instead of 106
- Q7: grouped by age instead of averaging it
- **All safe. All allowed by the guard. All wrong.**

Results — and why 10/10 is narrower than it sounds



Claude: first attempt on all ten, about 2 minutes and USD 0.40, every call logged.

What the 10/10 does not show

- Recorded on the pack before the semantic audit (29,500 characters); not re-run with Claude at delivery
- The pack names which entity answers which question — no hold-out set
- One sample per question; one key measure compared
- Honest claim: the layer is good enough for a strong model on these ten — not yet proof that it generalises

The ten answers, as delivered

Q	Question	Answer
1	Clients per region	south Moravia 937 · north Moravia 920 · … (client's own district, all 5,369)
2	Loans granted per year	1993 2.62 m (half year) … 1997 peak 30.73 m · total 103,261,740
3	Share of problematic loans	76 / 682 = 11.14% (B 31, D 45)
4	Average loan by owner's card	junior 221,970 (n = 21) · gold 161,881 · no card 148,865 · classic 148,809
5	Top districts, incoming 1997	Hl.m. Praha 115.13 m · Karvina 28.51 m · Ostrava 27.69 m …
6	New accounts per month 1996	106 · 104 · 124 · 101 · 91 · 113 · 137 · … · 125 (1,363)
7	Owner age at loan start	A 37.07 · B 39.60 · C 38.70 · D 37.00
8	Gold-card owners by gender	male 53 (60.23%) · female 35 (39.77%)
9	Average balance end Dec 1998	43,808.94 over 4,500 accounts (76 carried forward)
10	Frequency with highest problematic share	after transaction 18.75% (6 of 32 — highest observed, not significant)

What I would challenge myself on

1	**The month-end tie-break**	Highest trans_id picks the closing posting; right 0.4% of the time where the balance chain decides	*Disclosed · fixed after*
2	**Guard ≠ meaning**	A safe query can be wrong; the intent checker was a prototype	*Disclosed · fixed after*
3	**The 10/10 is narrow**	Old pack, question labels in the pack, no hold-out, one sample	*Partly disclosed*
4	**Disponent privacy**	Sees the owner's card; hidden values shown as 'no card'	*Found after delivery*
5	**Scope**	Well beyond a 1–2 day brief: 2FA, web page, deployment guide, agents	*Tasks first; what I'd cut*

My best-looking defect was my own bug

0.4%

how often the highest id was the true closing posting

55 → 14

material balance breaks once the order is read from the balance chain

+1.53

CZK change in Q9's average (43,808.94 → 43,810.47)

107

accounts with a different closing balance, up to 1,200 CZK

Account 3888, 31 Jan 1998	trans_id	Amount	Balance after
Interest credited	3,467,643	+1.30	10,501.30
Cash withdrawal — the real last posting	1,137,901	−1,200.00	9,301.30

Each posting starts at balance_after − amount, so a day's postings form a chain; its end is the closing posting. The source did record the order.

What I learned

My report said this account was 'off by exactly 1,200 — a missing posting'. It was my tie-break reading the balance before the withdrawal. A derived check inherits the bugs of what it reads; and 'unknowable' is a hypothesis to test.

My checks caught me — twice before delivery

The self-test nearly deleted real data

Unscoped quarantine view: injected defects excluded real rows — 76 districts instead of 77. The grain checks failed before any deliverable was written.

The filter leaked through one view

Three curated views were not security_invoker: ai.loan returned 682 loans to a user entitled to none. Found by logging in and counting.

The demo's own query was wrong

sum(DISTINCT amount) as a reflex guard against fan-out dropped 37 loans that share an amount: −4,480,920 CZK. Now pinned by a test.

A documented control without a test

The lockout was documented for weeks; writing its test at delivery found the rollback gap. Now a strict expected failure.

After delivery: the two fixes, on a separate branch

Not merged into the submission: D028 changes a certified number, and that needs the business to sign off the rule.

D028 · closing posting = end of the balance chain

	Delivered	After
Q9 average	43,808.94	43,810.47
Material breaks	55	14
Rounding band	81	70
Ambiguous days marked	—	42 at book end

Tested on synthetic postings and against an independent derivation: 180,000+ balances, 0 disagreements.

D029 · verify the intent, not only the shape

- The model states metric, periods, grouping and filters next to its SQL
- Checked before any database session: every column, the date filter day by day, no fan-out joins
- Then EXPLAIN, run under RLS, recompute the certified metric — rows only if both agree
- The gender query is refused, naming owner_gender; 10/10 and 4 new questions 4/4, verified

What breaks at 100,000× the volume

What breaks	Why	What it becomes
Rebuilds	core is CREATE TABLE AS every run: hours rewriting unchanged data	Month-partitioned facts, merge not replace
Rule runs	124 full scans	Partition-scoped rules; info sampled, block exact
Month-end fact	32 bn rows rebuilt from scratch	Maintained from last month + new postings
Latency	Ad-hoc SQL over 10 TB; one node; role-per-user pools	Pruning on certified time columns, marts, a gateway doing SET ROLE
Entitlement join	Millions of accounts per large portfolio	A materialised (role, account) set, co-partitioned
Evaluation	Ten questions are not a test set	Hold-out set, frozen samples, monitoring

What does not break: where the filter sits, onboarding as an insert, and the size of the semantic layer.

Decisions that are not mine to make

Problematic loan = status B or D	76 of 682 = 11.14% (A4)
Interest counts as incoming money	0.85% of 1997 incoming; same top ten (A7)
Calendar, not fiscal, quarters	There is no fiscal calendar in the source
Repeated identical interest postings	1,081 groups, 157,289.70 CZK: duplicates or not?
Every amount is CZK	No currency column (A19)

Each is written down with what changes if it is wrong, and implemented in one place so it can change in one place.

Every number traceable, nothing silently repaired, the AI never trusted

The filter sits where the model's text cannot reach it. The weak spots are written down — with the evidence and the fix.

Backup · the decision register in one line each

D002	PostgreSQL: RLS makes Task 3 real
D003	Verified public mirror; counts match
D004	Seed defects in a separate schema
D005	Claude CLI as a sandboxed completion
D006/17	One code map, code beside label, checked
D007	Natural keys: one source, no history
D008	Scope the quarantine to stg
D009	Filter in the engine; identity = session_user
D010	Every view security_invoker
D011	Value-based evaluation
D012	Rebuilding core restores the ai surface
D013	Never repair in place
D014	Placeholders → NULL, original kept
D015	19xx years; failed parse = NULL
D016	Gender and birth date derived, labelled
D018	Sign resolved once (VYBER is a debit)
D019	Month-end decided once (tie-break)
D020	Graded, not gated
D021	A customer IS one client
D023	Customers see what they may operate
D024	A disposition grants the account, not the person
D025	Three identifiers, three exposures
D026	2FA enforced by the database
D027	Derive grants; store only human decisions

Backup · security residuals, stated before they are asked

Residual	Status	Fix
Direct client rolls back failed second-factor guesses	Disclosed (strict xfail)	Count attempts outside the caller's transaction, or gateway-only logins
Session bound to backend PID only	A45	Compare backend_start too
Disponent sees the owner's card and disposition row	Found after delivery	Holder-based card policy; own-row dispositions; tests
Hidden owner fields shown as 'no card' / NULL	Found after delivery	No coalesce over RLS-hidden joins; say 'not visible'
TOTP secrets recoverable; demo shows codes	A44, A46	Encrypt with an external key; real enrolment
No read audit, no field masking, role-per-user pooling	Recommended	Audit log; masking; pool per role or SET ROLE gateway

Backup · Q9 under different closing-posting rules

Rule for the closing posting of a busy day	Average at 1998-12-31
Highest trans_id (delivered)	43,808.94
Balance chain, lowest id where ambiguous (D028, after delivery)	43,810.47
Balance chain, highest id where ambiguous	43,810.73
Lowest trans_id only	43,808.90

Typical wrong answer	Rows	Average
Join on max(date) without a tie-break	4,707	43,508.31
Only accounts with a December posting	4,424	44,209.53
Average of every December posting	23,553	46,908.00

What to say

The average is robust to the rule — within 2 CZK. Individual balances are not: 107 accounts differ. That is why the rule had to be fixed, not the average.